

搜索引擎项目

peanutixx

2026 年 7 月 29 日

Contents

1 第一期：离线部分

1.1 关键字推荐

1.2 网页搜索

2 第二期：在线部分

2.1 服务器框架

2.2 关键字推荐

2.3 网页搜索

3 第三期：缓存设计

3.1 缓存中应该存放什么类型的数据？

3.2 选择合适的缓存淘汰算法

3.3 如何解决并发访问的问题？

1

1

7

13

13

14

15

16

16

16

17

1 第一期：离线部分

1.1 关键字推荐

需求

根据语料库¹和停用词²生成词典库³和索引库⁴。

英文词典库和索引库：

dict_en.dat	index_en.dat
1 aah 1	1 a 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
2 aaron 355	2 b 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
3 aaronites 2	3 c 5 6 31 35 36 42 43 44 45 46 47 51 58 62 93 105 106 154 155 1
4 ab 2	4 d 7 10 11 12 13 14 17 19 22 32 33 34 35 36 37 38 39 40 41 42 4
5 aback 3	5 e 3 11 13 16 17 18 19 21 22 23 25 26 30 33 35 36 37 38 39 43 4
6 abacus 1	6 f 553 627 628 629 630 631 632 633 634 635 636 637 638 639 640
7 abaddon 1	7 g 8 12 20 44 49 66 78 83 97 112 123 137 146 147 148 150 170 17
8 abagtha 1	8 h 1 8 19 51 53 55 60 61 62 63 64 65 66 69 71 72 77 84 85 86 87
9 abana 1	9 i 3 12 15 20 30 34 35 36 37 40 44 45 52 54 55 66 67 68 69 70 7
10 abandon 32	10 j 87 88 105 106 107 455 456 457 458 459 460 461 462 463 464 46
11 abandoned 72	11 k 5 320 321 322 323 324 325 326 520 522 524 544 794 859 860 86
12 abandoning 27	12 l 33 37 40 50 51 52 53 54 55 70 79 83 84 89 90 91 92 93 99 100
13 abandonment 15	13 m 13 15 18 23 38 39 40 51 52 53 54 55 88 92 93 95 96 99 114 11
14 abandons 2	14 n 2 3 7 9 10 11 12 13 14 18 20 23 30 38 39 40 41 44 45 49 56 5
15 abarim 4	15 o 2 3 7 10 11 12 13 14 27 28 29 30 38 39 40 41 45 46 47 49 53

¹语料库 (Corpus): 是指经过科学收集和加工、能够代表某种语言或语言变体的真实文本 (或语音) 电子数据库。

²停用词: 是指在信息检索、文本分析或自然语言处理中, 被预先过滤掉 (忽略) 的词语。这些词通常极其高频, 但承载的信息量很低, 对理解文本的核心主题或情感贡献很小。

³在自然语言处理和语言学中, 词典库通常指一个包含词语和它语言属性的结构化信息的集合。常见的语言属性有: 词性、词频、释义、情感倾向等等。

⁴索引库: 是信息检索系统 (如搜索引擎、数据库) 中, 为快速查找文档而构建的一种高效数据结构。

中文词典库和索引库:

dict_cn.dat	index_cn.dat
1 一 1	1 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
2 一万年 1	2 丁 209 210 211 1603 2085 6977 8301 16774
3 一下 13	3 七 10 212 213 214 215 216 217 218 219 220 221 222
4 一下子 2	4 万 2 228 229 230 231 232 233 234 235 236 237 238
5 一世 2	5 丈 229 250 5445
6 一丝 2	6 三 251 252 253 254 255 256 257 258 259 260 261
7 一个 280	7 上 325 326 327 328 329 330 331 332 333 334 335
8 一个个 1	8 下 3 4 154 366 367 368 369 370 371 372 373 374
9 一举 1	9 不 178 298 367 393 394 395 396 397 398 399 400
10 一九二七年 1	10 与 649 650 651 652 653 654 655 656 657 658 659
11 一书 1	11 丑 663 664 665 666 2710 5752 6155 13375
12 一五 1	12 专 667 668 669 670 671 672 673 674 675 676 677
13 一些 72	13 且 692 693 842 6625 9497
14 一人 1	14 世 5 694 695 696 697 698 699 700 701 702 703 704
15 一代 5	15 丘 722 10528 12912

英文: 语料库位于 `corpus/EN` 目录, 停用词列表放在 `stopwords/en_stopwords.txt` 文件中。

1. 打开目录, 读取语料文件 (目录流: `opendir()`, `closedir()`, `readdir()`)。
2. 分词: 将数字和标点符号替换成空格, 只保留字母; 将字母统一转换成小写; 按空白字符分割。
3. 过滤掉停用词。
4. 统计每一个单词 (Token) 的出现的频率。
5. 生成词典库和索引库。

中文: 语料库位于 `corpus/CN` 目录, 停用词列表放在 `stopwords/cn_stopwords.txt` 文件中。

1. 打开目录, 读取语料文件 (目录流: `opendir()`, `closedir()`, `readdir()`)。
2. 使用 `cppjieba` 分词。
3. 过滤掉停用词。
4. 统计每一个单词 (Token) 的出现的频率。
5. 生成词典库和索引库 (需要使用 `utfcpp` 库分割出一个一个汉字)。

设计

有思路的同学, 请一定要按照自己的思路去实现, 哪怕最终发现是错的。没有思路的同学, 可以参照下面的设计:

```
DirectoryScanner.h
1 #pragma once
2 #include <vector>
3 #include <string>
4
5 class DirectoryScanner
6 {
7 public:
8     /**
9      * 遍历目录 dir, 获取目录里面的所有文件名
10     */
11     static std::vector<std::string> scan(const std::string& dir);
12
13 private:
14     DirectoryScanner() = delete;
15 };
```

```
1 #pragma once
2 #include <cppjieba/Jieba.hpp>
3 #include <string>
4 #include <set>
5
6 class KeyWordProcessor {
7 public:
8     KeyWordProcessor();
9
10    // chDir: 中文语料库
11    // enDir: 英文语料库
12    void process(const std::string& chDir, const std::string& enDir);
13
14 private:
15    void create_cn_dict(const std::string& dir, const std::string& outfile);
16    void build_cn_index(const std::string& dict, const std::string& index);
17
18    void create_en_dict(const std::string& dir, const std::string& outfile);
19    void build_en_index(const std::string& dict, const std::string& index);
20 private:
21    cppjieba::Jieba tokenizer_;
22    std::set<std::string> enStopWords_;
23    std::set<std::string> chStopWords_;
24 };
```

cppjieba 库

1. 安装

cppjieba 是一个 header-only 库，我们需要将头文件放到 /usr/local/include 目录下，其中 limonp 文件夹位于 cppjieba/deps/limonp/include 目录。

```
cd cppjieba
sudo cp -r include/cppjieba/ /usr/local/include/
sudo cp -r deps/limonp/include/limonp/ /usr/local/include/cppjieba/
```

安装之后如下图所示：

```

he@he-vm:~$ tree /usr/local/include/cppjieba/
/usr/local/include/cppjieba/
├── DictTrie.hpp
├── FullSegment.hpp
├── HMMModel.hpp
├── HMMSegment.hpp
├── Jieba.hpp
├── KeywordExtractor.hpp
├── limonp
│   ├── ArgvContext.hpp
│   ├── Closure.hpp
│   ├── Colors.hpp
│   ├── Condition.hpp
│   ├── Config.hpp
│   ├── ForcePublic.hpp
│   ├── LocalVector.hpp
│   ├── Logging.hpp
│   ├── NonCopyable.hpp
│   ├── StdExtension.hpp
│   └── StringUtil.hpp
├── MixSegment.hpp
├── MPSegment.hpp
├── PosTagger.hpp
├── Prefilter.hpp
├── QuerySegment.hpp
├── SegmentBase.hpp
├── SegmentTagged.hpp
├── TextRankExtractor.hpp
├── Trie.hpp
└── Unicode.hpp

```

```

he@he-vm:~$ tree /usr/local/dict/
/usr/local/dict/
├── hmm_model.utf8
├── idf.utf8
├── jieba.dict.utf8
├── pos_dict
│   ├── char_state_tab.utf8
│   ├── prob_emit.utf8
│   ├── prob_start.utf8
│   └── prob_trans.utf8
├── README.md
├── stop_words.utf8
└── user.dict.utf8

```

除了头文件之外，cppjieba 还依赖一些词典文件。词典文件位于 `cppjieba/dict` 目录，我们需要将它移动到 `/usr/local/` 目录下，如上图所示。

2. 使用教程

cppjieba 有多种分词方式，其中最重要的有三种方式：MP (Maximum Probability 最大概率分词)，HMM (Hidden Markov Model 隐马模型分词)，Mix (混合模式：默认方式)。它们的区别如下：

模式	方法	特性
MP	<code>Cut(sentence, words, false)</code>	基于前缀词典构建词图，使用动态规划找最大概率路径 (精确分词)
HMM	<code>CutHMM(sentence, words)</code>	基于隐马模型，适合新词识别
Mix	<code>Cut(sentence, words)</code>	先使用 MP 分词，再用 HMM 识别 MP 无法识别的新词

下面我们写一个例子，演示一下这三种分词方法的区别：

```

cppjieba_cut_words.cc
1 #include <cppjieba/Jieba.hpp>
2 #include <iostream>
3
4 using namespace std;
5
6 void print_words(const string& title, const vector<string>& words)
7 {
8     cout << "[" << title << " ] ";
9     for (const auto& w : words) {
10         cout << w << "/";
11     }
12     cout << endl;
13 }
14
15 int main()
16 {
17     // 创建 Jieba 对象会读取 /usr/local/dict 目录下的词典文件，比较耗时

```

```

18 // 最佳实践：最好只创建一个 Jieba 对象
19 cppjieba::Jieba tokenizer;
20
21 string s = "金胖胖是一名杰出的计算机科学家，他今天来到了武汉天源迪科，让我们热烈欢迎金胖胖同学！";
22 vector<string> words;
23
24 // [MP]
25 tokenizer.Cut(s, words, false); // HMM = false
26 print_words("MP", words);
27 // [HMM]
28 tokenizer.CutHMM(s, words);
29 print_words("HMM", words);
30 // [MIX]
31 // tokenizer.Cut(s, words, true); // HMM = true
32 tokenizer.Cut(s, words);
33 print_words("MIX", words);
34 }

```



```

[MP] 金/胖胖/是/一名/杰出/的/计算机/科学家/，/他/今天/来到/了/武汉/天/源/迪/科/，/让/我们/热烈/欢迎/金/胖胖/同学/！/
[HMM] 金/胖/胖/是/一名/杰出/的/计算机/科学家/，/他/今天来/到/了/武汉/天源迪科/，/让/我们/热烈/欢迎/金/胖/胖同学/！/
[MIX] 金/胖胖/是/一名/杰出/的/计算机/科学家/，/他/今天/来到/了/武汉/天源迪科/，/让/我们/热烈/欢迎/金/胖胖/同学/！/

```

从上面的结果来看，很明显 MIX 分词方式是最好的。

utfcpp 库

1. 安装

utfcpp 也是一个 header-only 库，我们只需要将对应的文件放到 `/usr/local/include/` 目录下即可：

```

cd utfcpp-4.0.6/
sudo cp -r source /usr/local/include/utfcpp

```

安装之后如下图所示：

```

he@he-vm:~$ tree /usr/local/include/utfcpp/
/usr/local/include/utfcpp/
├── utf8
│   ├── checked.h
│   ├── core.h
│   ├── cpp11.h
│   ├── cpp17.h
│   ├── cpp20.h
│   └── unchecked.h
└── utf8.h

```

2. 使用教程

先来看第一个示例，它演示了如何拆分 utf8 字符。

```

                                     utfcpp_split_characters.cc
1 #include <iostream>
2 #include <utfcpp/utf8.h>
3
4 using namespace std;
5

```

```

6 int main()
7 {
8     string s = "你好，世界! !";
9
10    const char* curr = s.c_str();
11    const char* end = s.c_str() + s.size();
12
13    while (curr != end) {
14        auto start = curr;
15        // 将 it 移动到下一个 utf8 字符所在的位置
16        utf8::next(curr, end);
17        // 一个汉字会占用多个字节，因此需要用 string 来表示一个汉字
18        string character = string(start, curr);
19        cout << character << endl;
20    }
21 }

```

运行结果:



我们再来看另一个例子，它演示了如何获取字符的 Unicode 码点 (Codepoint)⁵。

```

utfcpp_codepoint.cc
1 #include <iostream>
2 #include <utfcpp/utf8.h>
3
4 using namespace std;
5 int main()
6 {
7     string s = "你好，世界! !";
8     // 获取 utf8 的起始迭代器和末尾迭代器
9     auto it = utf8::iterator<string::const_iterator>(s.begin(), s.begin(), s.end());
10    auto end = utf8::iterator<string::const_iterator>(s.end(), s.begin(), s.end());
11
12    for (; it != end; ++it) {
13        char32_t codepoint = *it; // 获取码点
14        cout << "U+" << hex << codepoint << endl; // 以十六进制格式输出
15    }
16 }

```

运行结果:

⁵码点就是字符集（如 Unicode）给每个字符分配的唯一数字 ID。它是逻辑概念，不是具体的存储方式。

```
Terminal
peanut@wd:Examples$ ./a.out
U+4f60
U+597d
U+ff0c
U+4e16
U+754c
U+ff01
U+21
```

如果我们拿到一个字符的 **Unicode** 码点，我们就能做很多事情，比如：比较字符的大小（比较 **Unicode** 码点的大小），判断字符的类别（是不是中文汉字）等等。

1.2 网页搜索

需求

我们要根据给定的语料 (在公司中，往往是爬取下来的网页，或者是公司内部的文档)，生成网页库、网页偏移库和倒排索引库。

网页库的格式如下所示：包含 `<doc>`, `<id>`, `<link>`, `<title>`, `<content>` 等标签。

```
<doc>
  <id>1</id>
  <link>http://scitech.people.com.cn/n1/2021/0217/c1007-32030067.html</link>
  <title>中国空间站首批航天员乘组正着重开展出舱活动等训练</title>
  <content>    中新社北京2月16日电（郭超凯）记者16日从中国载人航天工程办公室获
</doc>
<doc>
  <id>2</id>
  <link>http://scitech.people.com.cn/n1/2021/0212/c1007-32028847.html</link>
  <title>天问一号“奔火”“太空刹车”成功</title>
  <content>    2月10日19时52分，中国首次火星探测任务“天问一号”探测器实施捕获制
</doc>
<doc>
  <id>3</id>
  <link>http://scitech.people.com.cn/n1/2021/0211/c1007-32028637.html</link>
  <title>中国科研团队发布量子计算机操作系统</title>
  <content>    据新华社电（记者徐海涛）操作系统是管理计算机软硬件的“大管家”，
</doc>
```

网页偏移库是用来快速定位文档的，它个格式为: `<文档 id>`, `<偏移量>`, `<文档大小>`。

```
1 0 2681
2 2681 2749
3 5430 1921
4 7351 2440
5 9791 7166
6 16957 1558
7 18515 1149
8 19664 1149
9 20813 1141
10 21954 1150
11 23104 1150
12 24254 1150
13 25404 1150
14 26554 1150
15 27704 1150
```

倒排索引库是搜索引擎非常核心的一个数据结构，它的格式如下：`<关键字>` `<文档 id>` `<关键字在文档中的权重>` [`<文档 id>` `<关键字在文档中的权重>`]...

```

一一 23 0.0202283 49 0.0136804 62 0.0196741 1280 0.0359543 1287 0.0485953
一一列举 2198 0.025631
一一对应 1921 0.0170253 3333 0.00885305
一万 1122 0.0654998 1320 0.0194682 1470 0.0192643 4304 0.032548
一万个 1630 0.024991 2647 0.023272
一万亿 47 0.0525517
一万亿美元 2922 0.0195681
一万八千 1770 0.0591041
一万名 3798 0.0573116 3938 0.0950532
一万块 4017 0.0422778
一万多 1819 0.0779154
一万多个 3525 0.047755
一万多名 1231 0.012096
一万头 279 0.0136218
一万年 502 0.0465448 1686 0.0154775

```

流程



1. 使用 `tinyxml2` 提取语料库中的文档:

- 我们要为 `xml` 文件中每一个 `<item>` 标签生成一个对应的 `Document` 对象。
- 如果 `<item>` 中有 `<content>` 标签, 将 `<content>` 标签的内容作为 `Document` 的 `content`; 如果 `<item>` 中没有 `<content>` 标签, 那么将 `<description>` 标签的内容作为 `Document` 的 `content`; 如果 `<item>` 中也没有 `<description>` 标签, 那么忽略该 `<item>`。

2. 使用 `simhash` 库对文档去重。

- 计算每篇文档的 `simhash` 值, 根据汉明距离对文档去重 (我们认为汉明距离在 3 以内的文档, 是十分相似的)。

3. 根据去重后的文档, 生成网页库和网页偏移库。

4. 使用 `cppjieba` 提取去重后文档中的关键词。

5. 使用 `TF-IDF` 算法计算关键词在文档中的权重, 并归一化。

6. 生成倒排索引库。

设计

有思路的同学，请一定要按照自己的思路去实现，哪怕最终发现是错的。没有思路的同学，可以参照下面的设计：

```
----- DirectoryScanner.h -----
1 #pragma once
2 #include <vector>
3 #include <string>
4
5 class DirectoryScanner
6 {
7 public:
8     /**
9      * 遍历目录 dir, 获取目录里面的所有文件名
10     */
11     static std::vector<std::string> scan(const std::string& dir);
12
13 private:
14     DirectoryScanner() = delete;
15 };
-----
----- PageProcessor.h -----
1 #pragma once
2 #include <string>
3 #include <vector>
4 #include <set>
5
6 #include "cppjieba/Jieba.hpp"
7 #include "simhash/Simhasher.hpp"
8
9 class PageProcessor
10 {
11 public:
12     PageProcessor();
13     void process(const std::string& dir);
14
15 private:
16     /// 解析 dir 目录下的 xml 文件, 提取文档, 放入 documents_ 中
17     void extract_documents(const std::string& dir);
18     /// 依据 SimHash 算法对 documents_ 去重
19     void deduplicate_documents();
20     /// 构建网页库和网页偏移库
21     void build_pages_and_offsets(const std::string& pages, const std::string& offsets);
22     /// 构建倒排索引库
23     void build_inverted_index(const std::string& filename);
24 private:
25     struct Document {
26         int id;
27         std::string link;
28         std::string title;
29         std::string content;
30     };
```

```

31
32 private:
33     cppjieba::Jieba tokenizer_;
34     simhash::Simhasher hasher_;
35     std::set<std::string> stopWords_;    // 使用 set, 而非 vector, 是为了方便查找
36     std::vector<Document> documents_;
37     std::map<std::string, std::map<int, double>> invertedIndex_;
38 };

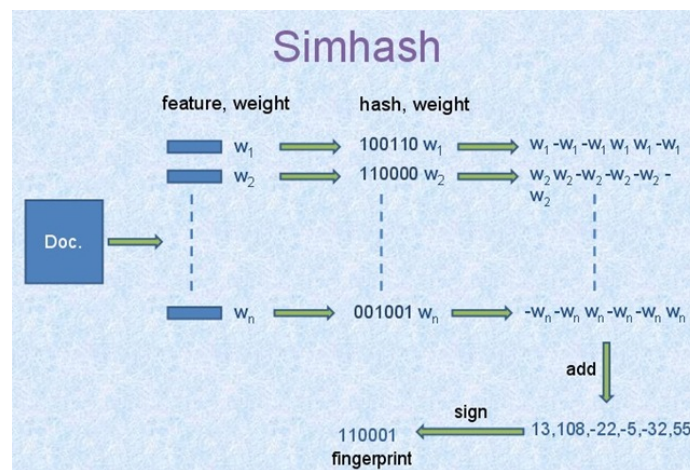
```

SimHash 算法

SimHash 是一种局部敏感哈希（Locality-Sensitive Hashing, LSH）算法，由 Google 在 2007 年提出，主要用于海量文本的近似去重和相似度检测。它的核心特点是：相似的文本生成相似的哈希值（海明距离小），不相似的文本生成的哈希值差异大。

原理

下面这张是 SimHash 经典的原理图：



它算法的流程如下：

1. 文本预处理与分词：对文档内容进行分词，并去除停用词。
2. 计算每个词的传统哈希值：使用普通哈希函数（如 MD5、SHA-1、FNV-1a 等）给每个词生成哈希值，但不直接使用这个哈希值作为结果。我们安装的 *simhash* 库，它使用的是 Jenkins 哈希算法。
3. 计算每个词语的权重。
4. 加权累加：按列相加。如果哈希码该列的值为 1，则加上权重；值为 0，则减去权重。
5. 二值化生成最终 SimHash 指纹：对累加结果 V 二值化处理，如果 $V[i] > 0$ ，则最终指纹的第 i 位为 1；反之，最终指纹的第 i 位为 0。

安装

simhash 库和 *cppjieba* 库是同一个作者写的，我们只需要将对应的头文件拷贝到 `/usr/local/include` 目录下即可。

```

cd simhash-1.3.3/include/
sudo cp -r simhash /usr/local/include/

```

安装成功之后，`/usr/local/include` 目录下会多一个 *simhash* 的文件夹：

```
he@he-vm:~$ tree /usr/local/include/simhash/
/usr/local/include/simhash/
├── jenkins.h
└── Simhasher.hpp
```

示例程序

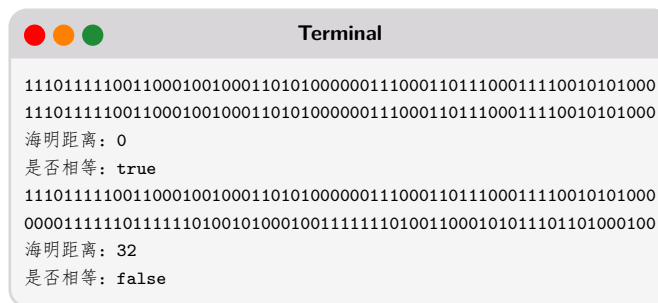
```
----- simhash_demo.cc -----
1 #include <bitset>
2 #include <ios>
3 #include <iostream>
4 #include <simhash/Simhasher.hpp>
5 #include <string>
6
7 using namespace std;
8 using namespace simhash;
9
10 // Simhasher 依赖 cppjieba, 创建对象时, 会读取词典文件, 比较耗时
11 // 最佳实践: 最好只创建一个 Simhasher 对象
12 Simhasher hasher;
13
14 int hamming_distance(uint64_t x, uint64_t y)
15 {
16     int distance = 0;
17     uint64_t z = x ^ y;
18     while (z) {
19         z &= (z - 1);
20         distance++;
21     }
22     return distance;
23 }
24
25 void print_similarity(const string& text1, const string& text2)
26 {
27     uint64_t h1, h2;
28     int topN = 5;
29     hasher.make(text1, topN, h1);
30     hasher.make(text2, topN, h2);
31     cout << bitset<64>(h1) << endl;
32     cout << bitset<64>(h2) << endl;
33
34     cout << "海明距离: " << hamming_distance(h1, h2) << endl;
35     // 默认情况下: 海明距离小于等于 3, 即为相等
36     cout << "是否相等: " << boolalpha << Simhasher::isEqual(h1, h2) << endl;
37     // 我们也可以像下面一样指定海明距离小于等于几, 才相等
38     // cout << "是否相等: " << boolalpha << Simhasher::isEqual(h1, h2, 6) << endl;
39 }
40
41 int main()
42 {
43     // =====
44     // 高度相似文档
```

```

45 // =====
46 string doc1 = "机器学习是人工智能的一个分支领域";
47 string doc2 = "机器学习是人工智能领域的重要分支";
48 print_similarity(doc1, doc2);
49
50 // =====
51 // 不相似文档
52 // =====
53 // 完全不同的主题
54 string doc4 = "机器学习是人工智能的一个分支领域";
55 string doc5 = "今天天气很好适合出门散步运动健康";
56 print_similarity(doc4, doc5);
57
58 return 0;
59 }

```

运行结果，如下所示：



```

Terminal
111011111000100100011010100000011100011011100011110010101000
111011111000100100011010100000011100011011100011110010101000
海明距离: 0
是否相等: true
111011111000100100011010100000011100011011100011110010101000
0000111110111110100101000100111111101001100010101101101000100
海明距离: 32
是否相等: false

```

不同大小的文本应该使用不同的 **topN**，一般来说：

文本长度（字节）	建议 topN
<= 600	5
~1200	10
~7800	65
>=24000	200

官方推荐使用如下规则获取 **topN** 的值：`max(5, min(200, text.size()/120))`。

TF-IDF 算法

TF-IDF (Term Frequency - Inverse Document Frequency, 词频-逆文档频率) 是一种用于信息检索与文本挖掘的常用加权技术。它的核心思想是：一个词在当前文档中出现的次数越多 (**TF** 高)，同时所有文档中出现的次数越少 (**IDF** 高)，那么这个词对当前文档就越重要。

该算法会涉及到如下几个概念：

- **TF (Term Frequency)**: 词语在文档中出现的频率。

$$TF = \frac{\text{词语在文档中出现的次数}}{\text{文档的总词数}}$$

- **DF (Document Frequency)**: 包含该词语的文档个数。

- IDF(Inverse Document Frequency): 逆文档频率, 其计算公式为

$$IDF = \log_2\left(\frac{\text{文档总数 } N}{DF + 1}\right)$$

理解这些概念之后, 我们就可以计算关键字的 **TF-IDF** 权重 w 了: $w = TF * IDF$ 。可以看出, 关键字的权重与它在文档中出现的次数成正比, 和包含该关键字的文档个数成反比。

一篇文档很可能会包含很多个关键字, 首先我们要计算每个关键字的 **TF-IDF** 权重: $w_1, w_2, \dots, w_n$, 然后归一化:

$$w'_i = \frac{w_i}{\sqrt{w_1^2 + w_2^2 + \dots + w_n^2}}$$

倒排索引中保存是关键字归一化后的权重, 即 w'_i 。

2 第二期: 在线部分

2.1 服务器框架

服务器端框架的选择有两种: **a) wfrest; b) muduo**。这里建议大家使用 **muduo**, 原因有以下几点:

- 相较于 **wfrest** (应用层), **muduo** 更为底层 (TCP 层), 给予开发者更多控制权, 便于定制优化。
- 目前 Linux 平台几乎所有的高性能网络框架都是基于 **Reactor** 模型, 学习和使用 **Reactor** 模型, 可以帮助大家更好地上手工作。
- 有助于理解 TCP 网络编程的理论知识, 达到更好的锻炼效果。

由于 **muduo** 是基于 TCP 的, 我们需要自定义协议来解决 TCP 的粘包问题, 通常情况下, 我们会采用 TLV 协议:

```
struct Message {
    uint8_t type;           // 消息的类型      1: 关键字推荐 2: 网页搜索
    uint32_t length;        // value 的长度
    std::string value;       // 消息的内容
};
```

编解码器:

```
// 协议格式: | Tag(1 字节) | Length (4 字节网络字节序) | Value (Length 字节) |
class TLVCodec {
public:
    using EntireMessageCallback = std::function<void(const muduo::net::TcpConnectionPtr&,
        uint8_t,
        const std::string&,
        muduo::Timestamp)>;

    explicit TLVCodec(const EntireMessageCallback& cb)
        : _messageCallback(cb)
    {
    }

    // 编码: 将消息打包成 TLV 格式发送
    void send(const muduo::net::TcpConnectionPtr& conn, uint8_t type, const std::string& value);
```

```

// 解码：从接收缓冲区解析出一条完整消息
void onMessage(const muduo::net::TcpConnectionPtr& conn,
muduo::net::Buffer* buf,
muduo::Timestamp receiveTime);

private:
    EntireMessageCallback _messageCallback; // 解析完成后的回调函数
};

```

服务器：

```

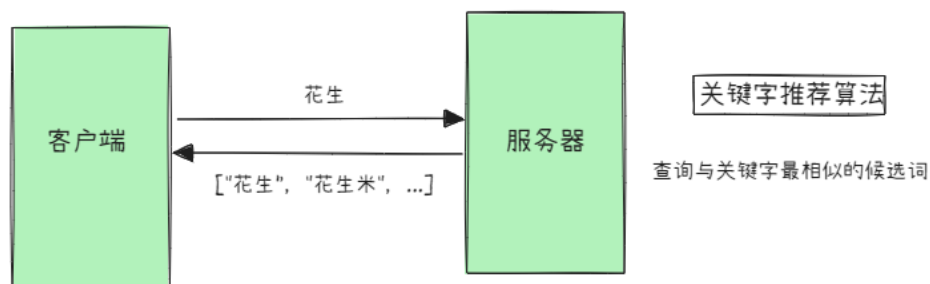
class SearchEngineServer {
public:
    SearchEngineServer(muduo::net::EventLoop* loop, const muduo::net::InetAddress& listenAddr);
    void start();

private:
    // 处理连接建立和断开事件
    void onConnection(const muduo::net::TcpConnectionPtr& conn);
    // 依据类型 (type), 调用 recommendKeywords 或 searchWebpages
    void onEntireMessage(const muduo::net::TcpConnectionPtr& conn,
        uint8_t type,
        const std::string& message,
        muduo::Timestamp receiveTime);
    // 关键字推荐
    void recommendKeywords(const muduo::net::TcpConnectionPtr& conn, const std::string& message);
    // 网页搜索
    void searchWebpages(const muduo::net::TcpConnectionPtr& conn, const std::string& message);

private:
    muduo::net::TcpServer _server;
    TLVCodec _codec;
};

```

2.2 关键字推荐



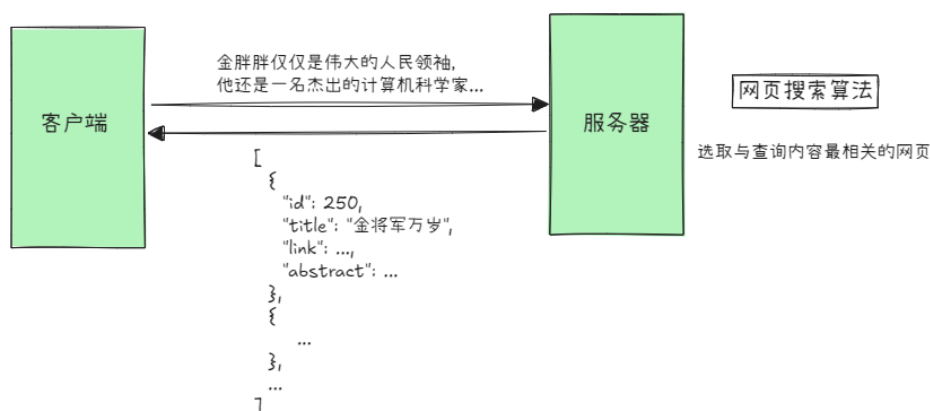
服务器收到客户端发来的关键字后，会根据索引库和词典库，选取与关键字最相近的一些候选词，返回给客户端。

候选词的选取

1. 将关键字拆分成一个一个字符。

2. 通过索引库获取每一个字符对应的词语集合，合并所有的词语集合，得到候选词集合。
3. 通过编辑距离算法，计算候选词集合中的候选词与关键字的相似度。
4. 选择最相近的候选词：
 - 优先比较编辑距离。
 - 在编辑距离相同的情况下，比较候选词的词频；优先选择词频高的候选词。
 - 在词频相同的情况下，按字典序比较候选词的大小；优先选择字典序小的候选词。
5. 选择最相近的 k (比如：5) 个候选词，以 JSON 格式返回给客户端，（提示：可以使用优先队列 `priority_queue`）。

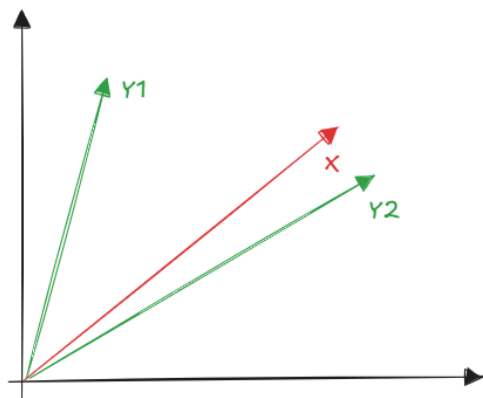
2.3 网页搜索



服务器接收客户端输入的查询内容，会根据倒排索引库、网页库和网页偏移库选取与查询内容最相关的一些网页，并根据网页内容（`content` 标签中的内容）生成摘要（`abstract`），返回给客户端。

网页的选取

1. 核心处理：将用户输入的查询内容视为一篇新的文档 x 。
 - 使用 `cppjieba` 对文档 x 进行分词，过滤停用词，获取关键字集合。
 - 使用 `TF-IDF` 算法计算出每个关键字的权重，将其组成一个向量 $X = (x_1, x_2, \dots, x_n)$ ，我们将该向量称为基准向量。
2. 通过倒排索引库，查询包含所有关键字的网页。当网页库很大的情况下，我们几乎总是可以找到这样的网页的；如果没有包含所有关键字的网页，我们就返回空数组。
3. 如果找到了包含所有关键字的网页，我们则采用余弦相似度算法对网页进行排序。
 - 既然网页包含所有的关键字，那么我们就可以通过倒排索引库获取每一个关键字的权重，将其组成向量 $Y = (y_1, y_2, \dots, y_n)$ 。
 - 计算基准向量 X 与 Y 的余弦值，该余弦值就代表了 X 与 Y 的相似度。
 - 根据余弦值对网页进行排序，余弦值越大越相似，排序越靠前。



$$\cos\theta = \frac{X \cdot Y}{|X||Y|}$$

$$X \cdot Y = x_1y_1 + x_2y_2 + \cdots + x_ny_n$$

4. 找到网页之后，我们需要提取每篇网页中的 `id`, `title`, `link` 信息，并根据网页内容生成摘要 (`abstract`)。将这些信息封装到 JSON 中，返回给客户端。生成摘要有两种方式：

- 静态摘要：将文档内容的前 `n` 个字符作为摘要，比如前 50 个字符。
- 动态摘要：将查询关键字附近的文字组成摘要 (注意：关键字区间重合的问题)。

3 第三期：缓存设计

我们可以使用缓存来加快查询结果。

在设计缓存之前，我们先回答下面几个问题：

3.1 缓存中应该存放什么类型的数据？

缓存的目的是加快程序运行的速度。一般来说，我们会把下面这些类型的数据放入缓存中：

- 读多写少的数据。这是缓存的黄金法则，数据被频繁读取，但修改频率很低。
- 计算耗时的数据（可以是计算的中间结果，也可以是最终结果）。
- 热点数据：在某段时间内被频繁访问的数据。
- 可以容忍短暂时间不一致的数据（最终一致性），比如“热门文章推荐”、“排行榜（非实时）”、用户评论数（允许几秒误差）。

具体到我们的项目中，我们可以缓存下面的数据（键值对）：

- 关键字 → 推荐结果，比如：“花生” → ["花生", "花生米", "花生壳", ...]
- 对于网页搜索功能，我们可以缓存热门文档的信息，避免频繁读取网页库。如：
文档 id → {"id": 250, "title": ..., "link": ..., "abstract": ...}。

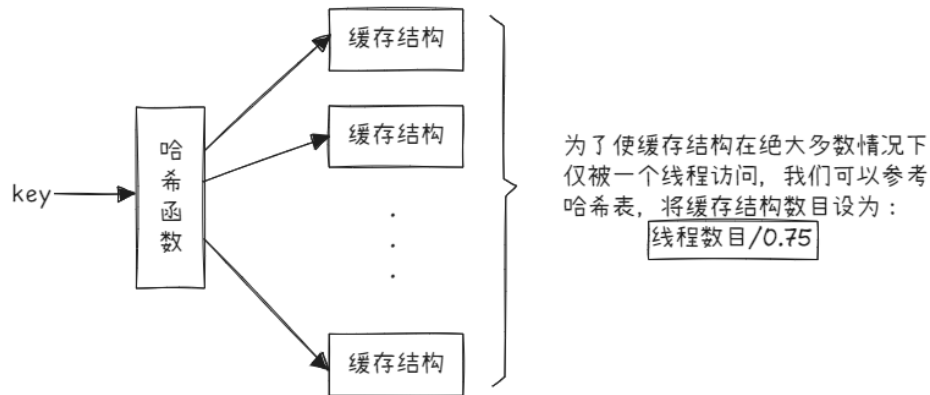
3.2 选择合适的缓存淘汰算法

- LRU：淘汰最久未被访问的数据。
- LFU：淘汰访问次数最少的数据。

- 其它的缓存淘汰算法，比如 LRU 的变种，LFU 的变种，或自适应替换缓存 (ARC) 等。

3.3 如何解决并发访问的问题？

假设有 n 个工作线程，它们都要访问缓存。如果缓存只有一个，那么所有线程都要争夺同一把锁，这样就会造成严重的排队等待问题。如果我们能够设计一个方案，使得：缓存结构在绝大多数情况下仅被一个线程访问，那么锁竞争导致的排队等待问题将迎刃而解。



下面是一个简单粗糙的设计 (仅供参考):

```
template<typename Key, typename Value>
class HashxxxCaches
{
public:
    void put(const Key& key, const Value& value);
    bool get(const Key& key, Value& value);
private:
    int capacity_;    // 总容量
    int slices_;     // 分片数量: cache 的数目
    std::vector<std::unique_ptr<xxxCache<Key, Value>>> _caches;
    std::hash<Key> hash_func_; // 哈希函数
}
```
